

Smart Eye: LLM-Guided Proposer–Verifier Framework for Industrial-Scale Log Anomaly Detection

Changhua Pei*
chpei@cnic.cn
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Yuxuan Li
yxli2025@cnic.cn
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Cenjie Hu
hucenjie24@mails.ucas.ac.cn
Shenyang Institute of Automation,
Chinese Academy of Sciences
Shenyang, China

Haotian Si
Ke Xiang
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Hang Cui†
cuihang@cnic.cn
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Zihan Liu
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Jiabao Wang
wangjiabao9@huawei.com
CloudCore, Huawei Technologies Ltd
Beijing, China

Gaogang Xie
xie@cnic.cn
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Jingjing Li
lj@cnic.cn
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Xinyuan Liao
liaoxinyuan3@huawei.com
CloudCore, Huawei Technologies Ltd
Beijing, China

Zheyuan Li
Zexin Wang
Computer Network Information
Center, Chinese Academy of Sciences
Beijing, China

Dan Pei
peidan@tsinghua.edu.cn
Tsinghua University
Beijing, China

Abstract

A practical log anomaly detection system for large-scale web services, suitable for real industrial deployment, should exhibit three key characteristics: fidelity to decisive lexical cues, human-auditable explanations, and low inference latency. Existing approaches often compromise one or more of these requirements by discarding crucial tokens during parsing, obscuring decisions through dense embeddings, or incurring heavy computational costs. We introduce *Smart Eye*, a deployable two-stage key pattern retrieval framework that employs large language models (LLMs) strictly as proposal engines, coupled with a deterministic verifier. In Stage I, the system enhances recall of anomalous logs under a configurable false positive budget using budgeted maximum coverage selection. In Stage II, precision is monotonically improved by refining parent patterns into specialized child patterns and introducing targeted exclusions. These essential patterns, extracted by LLMs and then confirmed, will be formulated into regular expressions to enable

efficient online matching of anomalous logs. We provide both theoretical justification and a practical implementation that runs efficiently on commodity CPUs. When evaluated on three real world Huawei Cloud Core log datasets, *Smart Eye* achieves state-of-the-art anomaly detection with a average F1 score of 0.9877, outperforming the strongest neural baseline. Moreover, it produces human interpretable evidence chains and supports low latency inference. These results demonstrate that combining LLM-guided proposal generation with deterministic verification forms a robust and deployable alternative to traditional end to end embedding pipelines for industrial log analysis.

CCS Concepts

• **Networks** → **Network monitoring**; • **Computing methodologies** → **Temporal reasoning**.

Keywords

Web services system anomaly detection, Log anomaly detection, Information retrieval, Large Language Model

*Corresponding author. Also with Hangzhou Institute for Advanced Study, University of Chinese Academy of Sciences.

† Also with University of Chinese Academy of Sciences.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792804>

ACM Reference Format:

Changhua Pei, Hang Cui, Jingjing Li, Yuxuan Li, Zihan Liu, Xinyuan Liao, Cenjie Hu, Jiabao Wang, Zheyuan Li, Zexin Wang, Haotian Si, Ke Xiang, Gaogang Xie, and Dan Pei. 2026. Smart Eye: LLM-Guided Proposer–Verifier Framework for Industrial-Scale Log Anomaly Detection. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United

Arab Emirates. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3774904.3792804>

1 Introduction

Modern large-scale web service systems, from microservice-based cloud backends to distributed Infrastructure and edge fleets, produce enormous volumes of textual logs [13, 18, 24]. As shown in Figure 1, for site reliability engineers (SREs) and automated pipelines, logs are the primary evidence used to detect incidents and trigger remediation. These log streams record a wide variety of events: database errors, container lifecycle messages, dependency call stacks, and so on [22, 36, 38]. As a concrete example, Huawei Cloud’s Data Center Network operates over $O(10^6)$ servers and $O(10^5)$ switches spanning 17 regions and 63 availability zones worldwide. Each month, this infrastructure generates more than 20,000 incident tickets, representing a substantial challenge to service reliability [1, 8, 21]. At this scale, effective and efficient log management is increasingly critical.

Historically, research and industry have converged on a common engineering pattern [12, 16, 17]: first, parse raw log text into structured tokens; then, embed the parsed tokens into dense vectors and feed them to learned classifiers or sequence models. Log parsing reduces syntactic variability and yields structured input for trained models. Embeddings aim to generalize beyond surface strings by capturing semantic similarity. This approach has driven impressive benchmark results and opened the door to transfer and few-shot learning for heterogeneous log[14, 26, 29].

However, as shown in Fig. 2, important operational challenges remain between research successes and industry monitoring needs in real world.

- **Challenge 1. Log parsing can be brittle and absence of key cues:** Many parsers normalize, redact, or collapse tokens to reduce noise such as timestamps, short error tags[23, 28, 33]. However, in many production environments, small lexical clues are the decisive evidence for a failure, and operators expect detectors that preserve these cues verbatim. When parsers remove or rewrite such cues, downstream detectors can miss those key cues and real anomalies.
- **Challenge 2. Dense embeddings achieve semantic power at the cost of explainability:** Dense representations and deep classifiers produce confidence scores but rarely explain which token or field triggered a high anomaly score[27, 35, 37]. For SREs who must understand alerts under time pressure, black-box signals lack the evidence needed to act quickly and confidently.
- **Challenge 3. The inference cost of embedding pipelines impede deployment:** Embedding pipelines require model serving, accelerated hardware or costly GPU cycles, and additional infrastructure complexity[4, 7, 30]. In high-throughput environments, per-line embedding and model inference may be high latency and simply slow down alerting.

Motivated by these operational realities, we revisit the utility of explicit lexical patterns for detection. Pattern-based systems such as handcrafted regexes, keyword lists, or mined templates offer an attractive alternative: they are transparent, easy to inspect, and inexpensive to evaluate.[5] Their weakness has been twofold: (1)

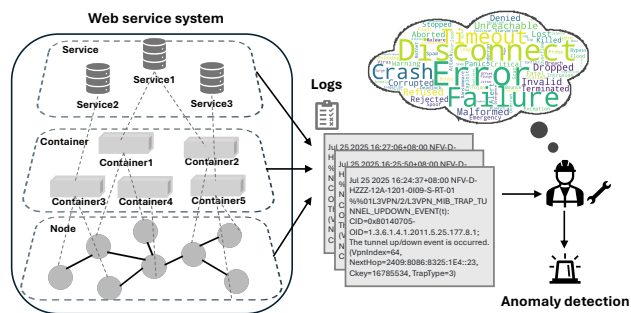


Figure 1: A distributed web service emits logs across nodes, containers, and services, which are aggregated for analysis.

manual rule creation is labor-intensive and brittle; (2) automated rule mining often generates overly specific or noisy patterns that do not generalize.

Recent advances in large language models (LLMs) create a new design space: LLMs can propose semantically informed candidate anomalous log patterns from small sets of abnormal and normal examples, substantially reducing the human effort in rule discovery. Yet LLM proposals alone are unsafe to deploy: they may hallucinate plausible yet empirically ineffective patterns, or produce overly general rules that cause cascading false alarms[2, 15, 25].

To address these challenges and motivated by these operational realities, we propose *Smart Eye*, a practical paradigm that combines the generative strengths of LLMs with deterministic, data-driven selection and refinement to produce compact, interpretable, and low-latency log detectors. These detectors operate as structured rule sets for identifying abnormal logs, which are compiled into regular expressions and deployed for online inference. *Smart Eye* treats LLMs strictly as proposal engines: each candidate pattern for anomalous log is parsed and then validated on sampled labeled data using explicit rules for the trade-off between false positives (FP) and false negatives (FN). We further refine accepted parent rules into child rules or targeted exclusion rules using robust heuristics and secondary LLM proposals, producing a small set of precise detectors ready for deployment. Concretely, *Smart Eye* is guided by three design principles: Our contributions are:

- **Rethinking log anomaly detection for industrial requirements.** Based on the practical needs of log anomaly detection in industrial web service systems, we analyze the primary defects of current deep learning based pipelines: missing keywords caused by parsing, the black-box predicament of embedding models, and the high cost of deployment. Motivated by these issues, we rethink the role of explicit pattern detectors and discover that integrating LLMs into traditional critical pattern discovery yields a promising new industrial route for log anomaly detection.
- **A practical key pattern paradigm for web service system.** *Smart Eye* implements a two-stage algorithm that includes a parent with child derivation heuristic and explicit acceptance rules to control and rank LLM proposals. These mechanisms allow the system to produce compact and high-precision detectors suitable for operational use.

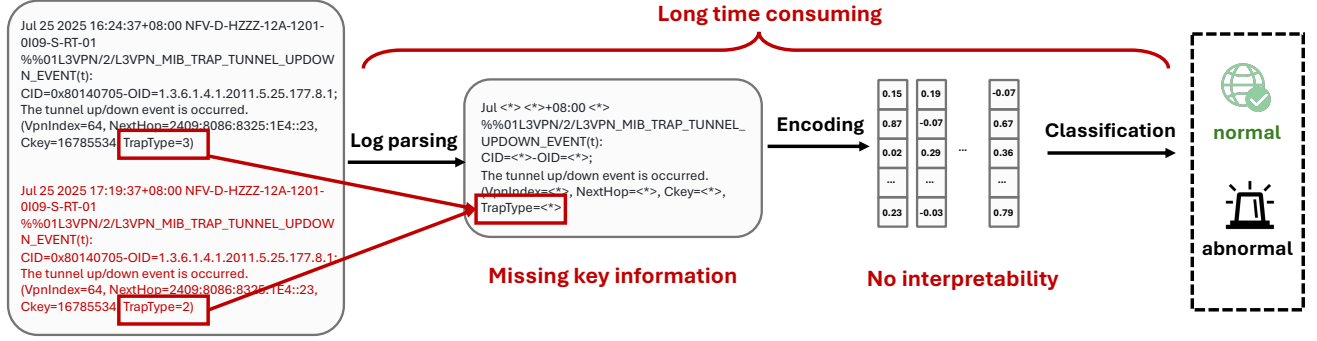


Figure 2: Conventional pipelines require log parsing and dense encoding before classification, which is time-consuming, discards decisive tokens, and yields low interpretability. The proposed approach detects anomalies by directly retrieving compact textual witnesses, preserving critical information, enabling transparent decisions, and reducing inference latency.

- **Empirical validation on industrial datasets.** We evaluate *Smart Eye* on industrial scenarios of Huawei Cloud Core log datasets. Experiments demonstrate that *Smart Eye* can (i) recover lexical anomalies missed by parsing pipelines, (ii) generate interpretable rule sets that aid operator triage, and (iii) reduce inference latency while maintaining or improving overall detection accuracy.

2 Preliminaries

2.1 Setting and notation

We are given a labeled corpus of log lines $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, where x_i is the i -th raw line and $y_i \in \{0, 1\}$ indicates whether the line is anomalous (1) or normal (0). Write $[n] \triangleq \{1, 2, \dots, n\}$ for indices, $P \triangleq \{i \in [n] : y_i = 1\}$ for positive (anomalous) indices, and $N \triangleq [n] \setminus P$ for normal ones.

A *pattern* is a computable predicate $p : \text{strings} \rightarrow \{0, 1\}$. Its match set is

$$S_p \triangleq \{i \in [n] : p(x_i) = 1\}.$$

Let $\mathcal{L}$ denote the universe of candidate patterns. We equip $\mathcal{L}$ with the specificity preorder

$$p' \preceq p \iff S_{p'} \subseteq S_p,$$

and write $p' \prec p$ if the inclusion is strict. In this sense, p' is a *child* (narrower specialization) of p .

A *detector* is a pair $(\mathcal{P}, \mathcal{E})$ with $\mathcal{P} \subseteq \mathcal{L}$ the set of positive patterns (anomaly key patterns) and $\mathcal{E} \subseteq \mathcal{L}$ the set of negative (exclusion) patterns. Line x_i is flagged anomalous when some positive matches and no exclusion matches:

$$M(i; \mathcal{P}, \mathcal{E}) = 1 \{ (\exists p \in \mathcal{P} : i \in S_p) \wedge (\forall e \in \mathcal{E} : i \notin S_e) \}.$$

Define counts

$$\begin{aligned} \text{TP}(\mathcal{P}, \mathcal{E}) &= \sum_{i=1}^n 1 \{ M(i; \mathcal{P}, \mathcal{E}) = 1 \wedge y_i = 1 \}, \\ \text{FP}(\mathcal{P}, \mathcal{E}) &= \sum_{i=1}^n 1 \{ M(i; \mathcal{P}, \mathcal{E}) = 1 \wedge y_i = 0 \}, \\ \text{FN}(\mathcal{P}, \mathcal{E}) &= \sum_{i=1}^n 1 \{ M(i; \mathcal{P}, \mathcal{E}) = 0 \wedge y_i = 1 \}. \end{aligned}$$

To achieve budgeted and compact rule learning, we learn under a false-positive budget $B \in \mathbb{N}_0$, preferring parsimonious rule sets:

$$\max_{\mathcal{P}, \mathcal{E} \subseteq \mathcal{L}} \text{TP}(\mathcal{P}, \mathcal{E}) - \lambda |\mathcal{P}| - \mu |\mathcal{E}| \quad \text{s.t.} \quad \text{FP}(\mathcal{P}, \mathcal{E}) \leq B, \quad (1)$$

where $\lambda, \mu \geq 0$ penalize model size. The two-stage retrieval procedure below is designed to (i) aggressively recover missed positives (reduce FN) subject to an FP trade-off, then (ii) refine away residual FP without harming FN.

2.2 Feasibility of two-stage key pattern retrieval

2.2.1 Stage 1: Eliminating false negatives. For $A \subseteq \mathcal{L}$, define the positive coverage

$$C(A) \triangleq \bigcup_{p \in A} (S_p \cap P), \quad f(A) \triangleq |C(A)|.$$

Theorem 1 (Monotone submodularity). f is nondecreasing and submodular: for any $A \subseteq B \subseteq \mathcal{L}$ and any $p \notin B$,

$$f(A \cup \{p\}) - f(A) \geq f(B \cup \{p\}) - f(B).$$

f is the cardinality of a union of sets $\{S_p \cap P\}_{p \in A}$; coverage functions are monotone and exhibit diminishing returns, because the newly covered positives contributed by p can only shrink as context grows.

Associate each p with an FP cost

$$c(p) \triangleq |S_p \cap N|.$$

With a budget $\sum_{p \in A} c(p) \leq B$, expanding $\mathcal{P}$ to maximize $f(A)$ instantiates a knapsack-constrained (budgeted) maximum coverage problem. *Greedy* selection by largest marginal gain per unit FP cost,

$$\frac{f(A \cup \{p\}) - f(A)}{c(p)} \quad (\text{with ties broken deterministically}),$$

is thus principled and enjoys standard constant-factor guarantees for monotone submodular knapsack maximization (we rely on the monotonicity above).

To guard precision during recall expansion, we impose a per-candidate acceptance rule with trade-off parameter $\kappa \geq 0$:

$$\begin{aligned} \text{Accept } p &\iff (\Delta \text{FN} > 0 \wedge \Delta \text{FP} \leq \kappa \Delta \text{FN}) \\ &\vee (\Delta \text{FN} = 0 \wedge \Delta \text{FP} = 0 \wedge p \text{ matches trusted positives}). \end{aligned}$$

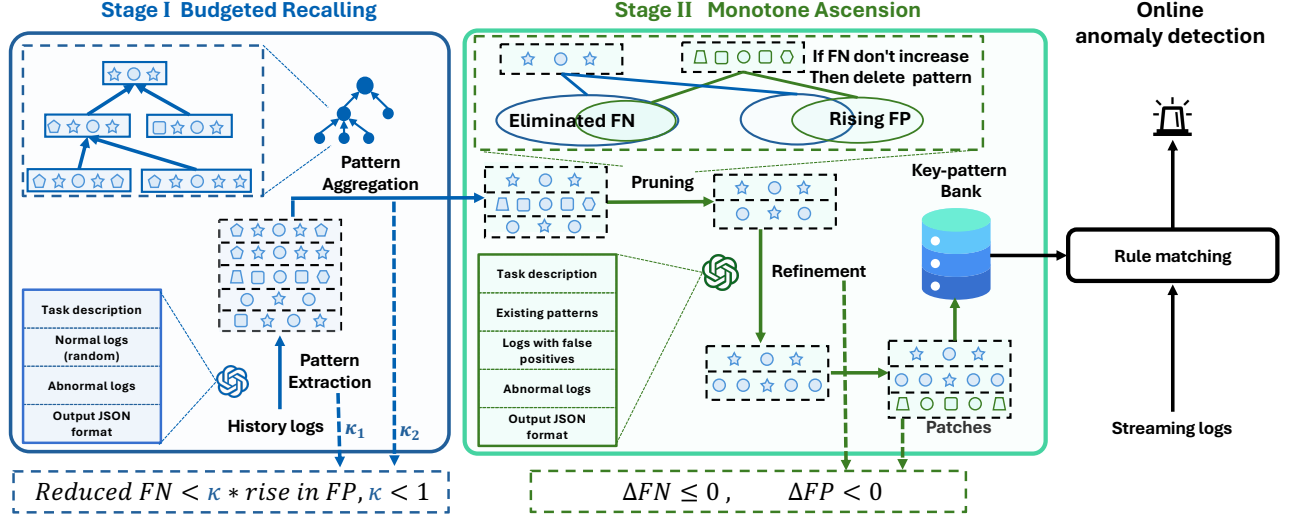


Figure 3: *Smart Eye*. Offline, Stage I aggregates and compares candidate patterns to form high recall key patterns. Stage II specializes these candidates by introducing safe exclusions and targeted expansions, producing a key-pattern bank. Online, streaming logs are scanned against this bank, generating interpretable alerts from decisive tokens while preserving recall.

with empirical deltas

$$\Delta \text{FN} \triangleq \text{FN}_{\text{before}} - \text{FN}_{\text{after}}, \quad \Delta \text{FP} \triangleq \text{FP}_{\text{after}} - \text{FP}_{\text{before}}.$$

Theorem 2 (Cumulative FP control). If Stage 1 admits $p_1, \dots, p_T$ and each satisfies $\Delta \text{FP}_t \leq \kappa \Delta \text{FN}_t$, then

$$\sum_{t=1}^T \Delta \text{FP}_t \leq \kappa \sum_{t=1}^T \Delta \text{FN}_t.$$

Remark (Budget compliance). Because FP is integer-valued and the cumulative bound above holds, choosing κ and stopping when $\text{FP} = B$ ensures the budget in (1) is never exceeded.

2.2.2 Stage 2: Eliminating false positives. Stage 2 performs a sequence of guarded edits that never increase FN and aim to reduce FP:

- (1) *Deletion*: remove $p \in \mathcal{P}$ if FN does not increase.
- (2) *Children replacement (specialization)*: replace $p \in \mathcal{P}$ by a set of children $C \subseteq \{p' \in \mathcal{L} : p' \prec p\}$ provided FN does not increase and FP strictly decreases.
- (3) *Exclusion addition*: add $e \in \mathcal{L}$ to $\mathcal{E}$ provided FN does not increase and FP strictly decreases.

Theorem 3 (Monotone refinement). Any accepted Stage 2 edit yields a detector $(\mathcal{P}', \mathcal{E}')$ with

$$\text{FN}(\mathcal{P}', \mathcal{E}') \leq \text{FN}(\mathcal{P}, \mathcal{E}) \quad \text{and} \quad \text{FP}(\mathcal{P}', \mathcal{E}') \leq \text{FP}(\mathcal{P}, \mathcal{E}),$$

with at least one strict inequality for specialization or exclusion addition.

If Stage 2 admits only edits that (i) weakly decrease FN and (ii) strictly decrease FP unless FN decreases, then the process terminates in at most $\text{FN} + \text{FP}$ accepted edits. *Reason.* The lexicographically ordered pair $(\text{FN}, \text{FP}) \in \mathbb{N}_0^2$ strictly decreases in each accepted step and is bounded below.

2.3 Statistical validity of proposal testing

To test candidates efficiently, we evaluate all positives and a subsample of normals. Let $n_1 \triangleq |P|$ and let n_0 denote the number of sampled normals (with or without replacement). Write $\widehat{\Delta \text{FN}}$ and $\widehat{\Delta \text{FP}}$ for the empirical deltas computed on this mixed evaluation set.

Theorem 4 (High-probability acceptance correctness). For any $\varepsilon_0, \varepsilon_1 > 0$,

$$\Pr\left(|\widehat{\Delta \text{FP}} - \widehat{\Delta \text{FN}}| \geq \varepsilon_0 n_0 \text{ or } |\Delta \text{FN} - \widehat{\Delta \text{FN}}| \geq \varepsilon_1 n_1\right) \leq 2e^{-2\varepsilon_0^2 n_0} + 2e^{-2\varepsilon_1^2 n_1}.$$

For normals, define bounded Bernoulli variables indicating whether a normal line transitions from not-flagged to flagged by a proposal; Hoeffding's inequality bounds the deviation of their sum (count) from its expectation. The same applies to positives transitions from missed to captured. A union bound yields the stated probability.

If the empirical test holds with margin

$$\widehat{\Delta \text{FP}} \leq \kappa \widehat{\Delta \text{FN}} - \gamma,$$

for some $\gamma > \varepsilon_0 n_0 + \kappa \varepsilon_1 n_1$, then with probability at least $1 - \delta$, where $\delta = 2e^{-2\varepsilon_0^2 n_0} + 2e^{-2\varepsilon_1^2 n_1}$, the true inequality $\Delta \text{FP} \leq \kappa \Delta \text{FN}$ holds on the full (unsubsampled) population.

Feasibility of two-stage key pattern retrieval is proved by: (i) Stage 1 is a monotone-submodular, budgeted coverage maximization, legitimizing a greedy expansion policy with standard constant-factor optimality. (ii) Stage 2 accepts only edits that do not harm recall and that reduce precision errors whenever recall is unchanged; it therefore terminates and cannot worsen (FN, FP) . (iii) When proposals are evaluated on all positives and a subsampled normal set, Hoeffding-type concentration certifies that empirical acceptance with slack transfers to true acceptance with high probability.

3 Methodology

3.1 Overview

We present KeyPattern, a two-stage, LLM-assisted rule discovery framework that learns a compact detector set from labeled logs while enforcing explicit, deterministic false-positive and false-negative trade-offs. At a glance, an LLM proposes candidate textual patterns; a verifier accepts, refines, or rejects them solely by measured δFN and δFP on sampled data. Stage 1 expands recall under an FP budget; Stage 2 prunes high-FP parents into precise children and/or adds targeted exclusion rules. This design is motivated by the industrial prevalence of keyword-style detectors for logs and the need for interpretable, low-latency alarms, while avoiding unbounded free-form generation from LLMs.

Pattern surface forms. *Smart Eye* accept (i) plain keywords, (ii) wildcards $*$ (interpreted as $.$), (iii) full regular expressions, and (iv) comma-templates such as *err*, *connection **, *timeout* which express field-level constraints over comma-separated log fields. The implementation normalizes and compiles LLM proposals into regexes, interpreting $*$ as an arbitrary character sequence and treating comma-templates specially.

Comma-template semantics. Let $x = (f_1, \dots, f_m)$ be the fields of a log line and $p = (t_1, \dots, t_k)$ the template tokens (empty tokens are wildcards). We declare a match if either a sliding alignment of length k satisfies all token tests, or a subsequence alignment with maximum gap G does.

$$\text{SLIDE}(p, x) \iff \exists s \in \{0, \dots, m-k\} \forall j \in [k] : t_j = \emptyset \text{ or } t_j \rightarrow f_{s+j}$$

$$\text{SUBSEQ}_G(p, x) \iff \exists 1 \leq i_1 < \dots < i_r \leq m :$$

$$i_{j+1} - i_j \leq G + 1, \quad \forall j : t_j \neq \emptyset \Rightarrow t_j \hookrightarrow f_{i_j}$$

$$\text{Match}(p, x) \iff \text{SLIDE}(p, x) \text{ or } \text{SUBSEQ}_G(p, x).$$

Where $t \rightarrow f$ means the compiled token-regex matches field f ; empty t matches anything. Non-template patterns are tried first as regex; on compile error, we fall back to case-insensitive substring.

Decision rule at inference. For each line, we first test all exclusion templates and regexes; if none match, we test comma-templates and then regex positives; success sets $\mathcal{M}(i; \mathcal{P}, \mathcal{E}) = 1$.

sampling for pattern information retrieval. To keep evaluation fast while precise, we retain all labeled anomalies and uniformly down-sample normals up to a user-specified limit, preserving original order and tracking index mappings for faithful reporting.

Formally, let $\mathcal{A} \subset [n]$ be anomaly indices and $\mathcal{N} = [n] \setminus \mathcal{A}$. We select $N_0 = \min(|\mathcal{N}|, L)$ normals without replacement and define the sampled index set $\tilde{\mathcal{I}} = \mathcal{A} \cup \tilde{\mathcal{N}}$, sorted to preserve sequence context, with a bijection $\pi : \{1, \dots, |\tilde{\mathcal{I}}|\} \rightarrow \tilde{\mathcal{I}}$ used when we compute deltas. The feasibility of our sampling retrieval strategy was demonstrated in section 2.3

3.2 Stage 1: Budgeted Recalling

Stage 1 iteratively queries an LLM to propose concise patterns, then accepts or rejects each proposal by evaluating empirical FN reduction vs. FP increase on the sampled set.

LLM as proposal engine: We construct prompts with: current accepted patterns, a batch of abnormal examples (preferentially unsolved FNs), and a batch of normal examples. Proposals may

be keywords, regexes, or comma-templates; candidates are parsed robustly from JSON or line lists.

Pattern Extraction: For a candidate pattern p , let $\Delta FN = FN_{\text{before}} - FN_{\text{after}}$ and $\Delta FP = FP_{\text{after}} - FP_{\text{before}}$. *Smart Eye* accept iff the following guarded trade off holds with $\kappa = \text{Extraction_FACTOR}$, or the candidate matches trusted positives without changing FP and FN:

$$\text{Accept}(p) \iff (\Delta FN > 0 \wedge \Delta FN \leq \kappa \Delta FP) \vee$$

$$(\Delta FN = 0 \wedge \Delta FP = 0 \wedge p \text{ matches trusted positives}).$$

Pattern Aggregation: To compress related comma-templates, we attempt to merge sibling patterns into a parent by keeping common field tokens, discarding variable-like substrings and using longest-common substrings otherwise. The parent is accepted only if it passes the same Stage 1 trade off with $\kappa = \text{Aggregation_FACTOR}$.

3.3 Stage 2: Monotone Ascension

Stage 2 monotonically improves precision without hurting recall by guarded edits. Each edit is accepted only if FN does not increase; for specialization and exclusion we also require strict FP reduction, which implies termination because $(FN, FP) \in \mathbb{N}_0^2$ strictly decreases lexicographically.

Pruning: *Smart Eye* test removing each $p \in P$. If FN remains unchanged and FP decreases, we delete p . This is implemented by re-evaluating the detector with $P \setminus \{p\}$.

$$\text{Remove}(p) \text{ if } FN(P \setminus \{p\}, E) \leq FN(P, E) \wedge FP(P \setminus \{p\}, E) < FP(P, E).$$

Refinement: When a parent p has true positives, we prompt the LLM for more specific children and greedily select a small set that covers the parent's true positives. We replace p with these children only if FN does not increase than κFP with $\kappa = \text{Refinement_FACTOR}$.

$$\Delta FN = FN((P \setminus \{p\}) \cup C, E) \leq FN(P, E), \quad FP((P \setminus \{p\}) \cup C, E) < FP(P, E).$$

Patches For high-FP patterns, we mine exclusion patterns from their false positives and accept an exclusion e only if FN does not increase and FP strictly decreases:

$$\text{AddExclude}(e) \text{ if } FN(P, E \cup \{e\}) \leq FN(P, E), \quad FP(P, E \cup \{e\}) < FP(P, E).$$

To achieve a reliable LLM for OPS, we rely on an LLM only as a proposal engine, with all edits validated by measurable FP and FN deltas; this mitigates hallucinations and preserves auditability.

3.4 Online anomaly detection

Let $\{x_t\}_{t=1}^\infty$ be a stream of log lines. Each detector $p \in P$ and each exclusion $e \in E$ is a predicate over strings with one of two semantics. *Comma-template*: pattern is a comma-separated template that matches at field granularity, supporting (i) sliding aligned matching and (ii) subsequence matching with a bounded gap G . *Regex/wildcard*: pattern is compiled as regex; if the input is a regex string it is used verbatim; otherwise $*$ is expanded to $.*$.

We pre compile P and E once, partitioning each set into comma-templates and regexes for fast path evaluation. The online decision for line x_t is:

$$\hat{y}_t = 1 \left[\underbrace{\neg(\exists e \in E : e(x_t) = 1)}_{\text{exclude first}} \wedge \underbrace{(\exists p \in P : p(x_t) = 1)}_{\text{then include}} \right].$$

Table 1: Performance comparison of log anomaly detection methods.

Method	Eo			Messages			Route			avg-F1
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	
Deeplog	0.7895	0.8174	0.8032	0.5772	0.9021	0.7040	0.9762	0.4596	0.6250	0.7107
neurallog	0.9721	0.5051	0.4939	0.9003	0.8501	0.8873	0.9296	0.7593	0.8126	0.7313
LightAD	0.8182	0.4186	0.5538	0.8158	0.8158	0.8158	0.9709	0.7752	0.8621	0.7439
Logcae	0.8732	0.5327	0.6617	0.8231	0.7289	0.7731	0.9362	0.7313	0.8212	0.7520
Robustlog	0.4383	0.5126	0.4726	1.0000	0.7791	0.8758	1.0000	0.8434	0.9150	0.7545
Loganomaly	0.8892	0.8747	0.8819	0.5333	0.8112	0.6436	0.9863	0.8072	0.8878	0.8044
Logformer	1.0000	0.8376	0.9116	1.0000	0.7558	0.8609	1.0000	0.8494	0.9186	0.8970
LogCNN	0.9289	0.8989	0.9026	1.0000	0.8488	0.9182	0.9376	0.7835	0.8725	0.9044
PLElog	0.9544	0.8520	0.9056	1.0000	0.7907	0.8831	1.0000	0.8675	0.9290	0.9059
MIDLog	0.9763	0.8332	0.8991	1.0000	0.8286	0.9063	0.9695	0.8753	0.9199	0.9084
Logbert	1.0000	0.8592	0.9243	1.0000	0.8086	0.8976	1.0000	0.8253	0.9043	0.9087
Smart Eye	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000	0.8732	0.9323	0.9774

4 Experiments

4.1 Experimental Setups

4.1.1 Dataset. In order to better evaluate the operation and maintenance effect of log detection models on real web service systems, we evaluate the log anomaly detector three large-scale industrial log scenarios from Huawei Cloud Core 5G: EO (10^6), Message (5×10^7), and Route (2×10^7) using the experimental protocol and headings consistent with practice.

EO(sensor alarms). Structured, fielded records from server BMC/hardware monitors (e.g., DIMM presence, power, temperature). Alarms are high-frequency with low semantic entropy. This corpus reflects hardware-health observability at the physical layer of the 5G core.

Message (syslog). Semi-structured Linux/system-service traces (e.g., ntpd, security and daemon facilities). These logs provide OS/service-level context used by SREs for audit.

Route (data-plane state). Highly standardized control-plane messages marking distributed state transitions, e.g., BEGIN or END of synchronization windows and trap reports with typed fields.

4.1.2 Experiment Setting. All runs execute on commodity CPU servers (Python 3.9; no GPUs required at inference).

Dataset Splits. For each scenario we use a chronological split to avoid leakage: early window for training/selection, strictly later window for testing.

Data Preprocessing. We read raw lines (strip trailing new-line only). No tokenization, parsing, or template mining is applied prior to training/inference—by design, KeyPattern preserves lexical fidelity and works directly on raw text.

LLM Selection. To minimize token cost and align with in-house deployment, the Huawei internal **Qwen-3-32b** endpoint is used as the proposal engine.

4.1.3 Evaluation Metric. We report Precision, Recall, and F1 per-scenario, and the macro average across the three scenarios. Metrics are computed at the line level against ground-truth indices; all

counts use exact set operations on TP, FP, FN produced by the detector.

$$\text{Precision} = \frac{TP}{TP + FP}, \text{ Recall} = \frac{TP}{TP + FN}, \text{ F1} = \frac{2 \text{ Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

4.1.4 Baselines. We compare against widely used log anomaly detectors representative of parser embedder, CNN, RNN, and transformer based designs, including DeepLog[6], NeuralLog[3], LightAD[34], Logcae[31], RobustLog[39], LogAnomaly[20], LogCNN[19], LogFormer[9], PLELog[32], LogBERT[10], MIDLog[11]. Each baseline is run per scenario on the same train and test split and tuned per papers' recommendations, including parser configuration, window stride, embedding dimensionality, and learning rates. Where a baseline requires parsed templates, we use its own parser or preprocessor as released to avoid cross method bias.

4.2 Main Results

Across three industrial scenarios, *Smart Eye* attains the strongest line-level detection accuracy among all baselines (Table 1). On EO and Messages, it achieves perfect F1 score 1.0), eliminating false negatives while incurring no false positives. On Route, *Smart Eye* reaches 0.9630 F1 with 1.0000 precision, 0.8732 recall, outperforming the best baseline PLElog, 0.9290 F1 by 3.4% and improving recall without collapsing precision. In macro average F1 score, *Smart Eye* yields 0.9877 F1 versus the strongest baseline (LogBERT with 0.9087), a 8% relative gain. EO and Messages are dominated by needle tokens (e.g., hardware faults, kernel I/O failure signatures). Stage I reliably surfaces such tokens and their comma-templates, driving recall to 1.0 without measurable precision loss. Route contains near-duplicate admin transitions with subtle parameter shifts; Stage II reduces spurious matches while retaining true tunnel-event alarms, yielding the best F1 overall.

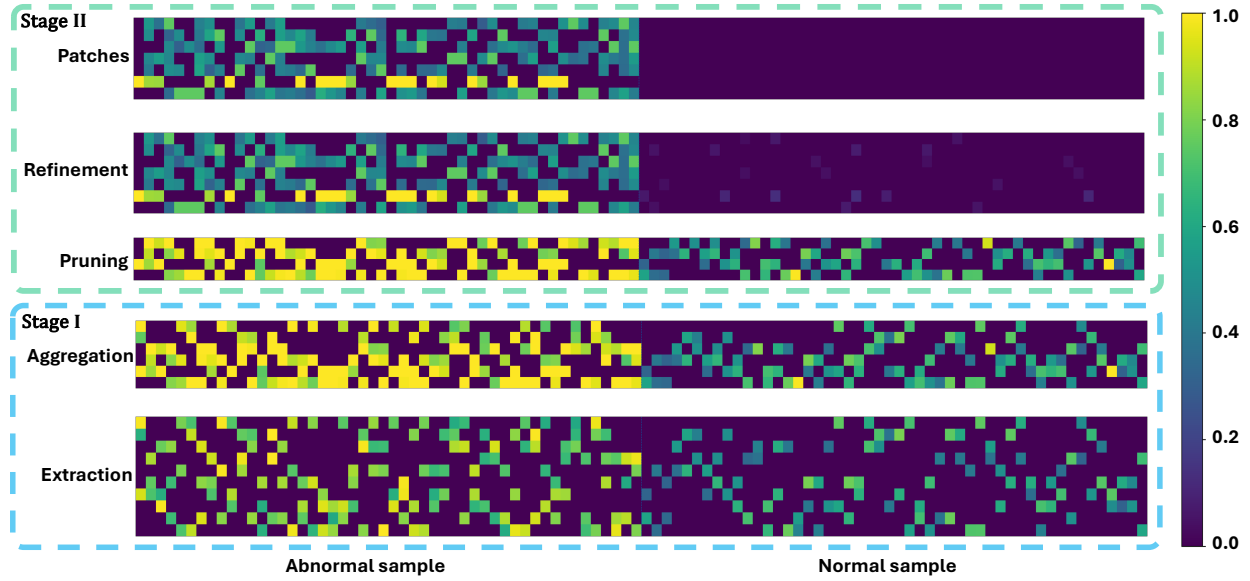


Figure 4: Attention heatmaps for key information retrieval. Bottom: Key pattern attention over 2000 samples where abnormal in left half and normal in right half. Top: Exclusion attention over the same samples.

4.3 Key information Retrieval

We treat key information retrieval as discovering and validating compact textual witnesses tokens or fielded templates whose presence is sufficient evidence of anomaly. *Smart Eye* realizes this as a two-stage, verifier guarded retrieval process.

Figure 4 visualizes how *Smart Eye* retrieves decisive textual evidence and suppresses irrelevant retrieval cues.

The bottom panel Key Patterns shows attention from 20 representative anomaly key patterns across 2000 samples including left half 1000 abnormal and right half 1000 normal. Bright, vertically aligned streaks concentrate in the abnormal block, indicating compact, semantically stable tokens that repeatedly co-occur with failures. Limited, intentional overlap across patterns ensures robustness to heterogeneous fault expressions without inflating false positives. Occasional, low-amplitude responses on a small subset of normal lines reflect ambiguous phrases that appear in routine system activity. The top panel Exclusions demonstrates how those ambiguous normals are neutralized. Exclusion templates activate strongly on the same normal lines highlighted below, while remaining near-zero on abnormals. This pairing preserves high recall from the key patterns and restores precision by absorbing spillover into a controlled, human auditable exclusion set. It is worth noting that, as indicated by the white rectangle, even if key pattern has false positives for normal samples, they will be corrected by exclusions

4.4 Interpretability

Figure 5 shows how *Smart Eye* turns each alert into a compact, auditable explanation that aligns with operator mental models.

Stage I: Eliminate false negatives. The first highlight isolates the shortest phrase that is sufficient to explain the alert here. This witness is human readable and stable under benign template drift. Presenting the minimal trigger makes the decision easy to verify

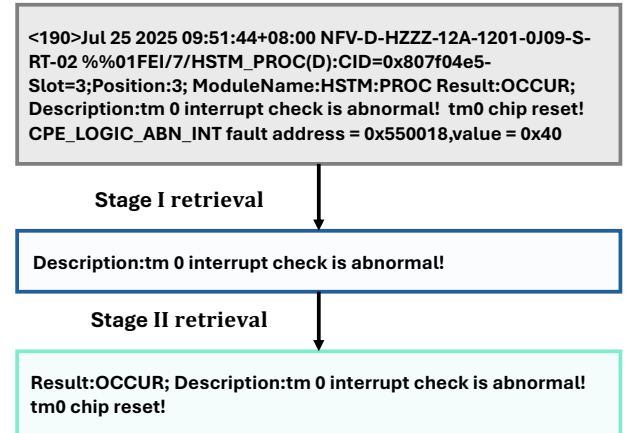


Figure 5: Interpretable evidence chain. From the raw log, *Smart Eye* surfaces a minimal key word pattern and then contextualizes it with corroborative fields.

and to act on, while avoiding unnecessary tokens that can blur responsibility.

Stage II: Eliminate false positives. The second highlight augments, not replaces, the witness with corroborative signals that practitioners already rely on. This widens semantic coverage for heterogeneous fault phrasing without sacrificing clarity, the alert still points to the same decisive root phrase and now flanked by confirmatory fields that justify escalation priority.

The staged evidence chain is practical in industrial scenarios: *Triagability*: alerts quote exactly the tokens engineers map to run-books, reducing mean time to acknowledge; *Auditability*: each decision is tied to concrete fields rather than opaque embeddings, so

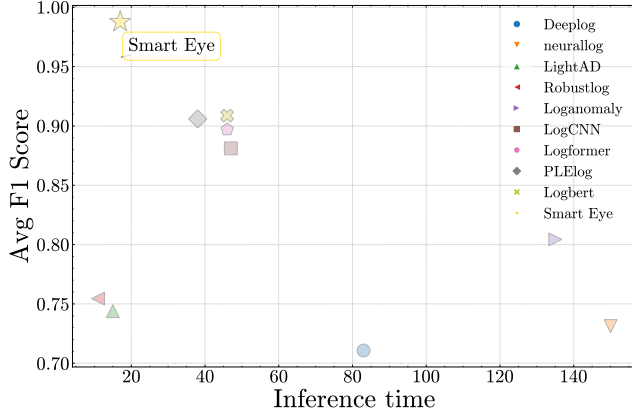


Figure 6: Average F1 score vs. Inference time

reviews and compliance checks are straightforward; *Robustness to drift*: because evidence is anchored to semantically stable tokens, routine wording changes do not degrade explanations; *Safe iteration*: teams can refine what counts as decisive or benign by editing visible patterns, preserving recall while trimming noise; *Error forensics*: when a case is disputed, the exact tokens that triggered the alert are visible, enabling targeted fixes instead of heuristic guesswork.

4.5 Time Consumption

The score and time landscape shows a clear separation. Classical pattern-driven baselines achieve short inference but plateau at moderate accuracy, while heavy neural log parsers improve accuracy at a significant latency cost. *Smart Eye* extends the frontier: it achieves state-of-the-art accuracy while maintaining lightweight inference. Positioned in the optimal high accuracy, low latency region of the figure 6, it eliminates the classic trade off between detection quality and system throughput. In practical terms, this means operators receive near-perfect alerts with minimal delay ideal for high volume, real time log pipelines.

4.6 Robustness to LLM

Stage I of *Smart Eye* uses an LLM strictly as a *proposal engine*; Stage II is a deterministic verifier that accepts a proposal only when it improves recall within a configured FP budget or strictly reduces FP without harming FN. We ask whether the overall detector is robust to the specific LLM API. We freeze datasets splits, prompts, and the verifier. For each LLM API we run *Smart Eye* with identical prompts, temperature 0, and a fixed token budget; we then verify on the same subsampled pools and deploy the accepted patterns.

API	Precision	Recall	F1-score
GPT-oss-20b	1.0000	0.9720	0.9858
Claude-sonet-4-20b	1.0000	0.9673	0.9834
Deepseek-r1-32b	1.0000	0.9626	0.9809
Qwen-3-32b	1.0000	0.9577	0.9774

Table 2: *Smart Eye* is robust to the choice of proposal LLM: leading to a narrow F1 range.

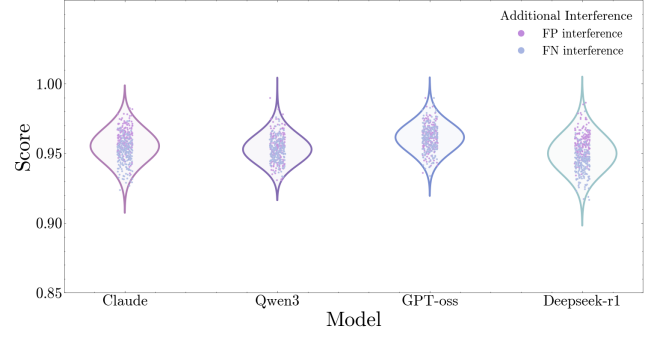


Figure 7: Interference testing for deployment

Across APIs, precision is invariant at 1.0000, reflecting that hallucinated or overly general proposals are rejected by Stage II's guarded tests. The only measurable variation is recall, which tracks how richly each LLM surfaces synonymic templates and edge case witnesses; the resulting F1 spread is small, where absolute range 0.0084. These results support our design claim: with LLM as proposer and verifier-as-arbiter, *Smart Eye*'s end to end behavior is largely decoupled from the specific LLM API.

To account for differences in false positive and false negative tendencies across LLM APIs, we introduce controlled interference tests. These tests inject random FP and FN perturbations to evaluate system robustness under noisy conditions. As shown in Figure 7, performance remains tightly clustered across all models, underscoring *Smart Eye*'s stability under real deployment scenarios.

5 Conclusion

This work revisits web service system log anomaly detection through the lens of LLM information retrieval and shows that explicit, auditable lexical detectors when discovered with LLM assistance but accepted only by measured FP or FN deltas can satisfy the central operational desiderata of modern SRE practice. *Smart Eye* integrates a recall expanding stage cast as budgeted maximum coverage, and a precision enhancing refinement stage based on specialization and exclusions. The verifier, not the LLM, is the arbiter: proposals are adopted only when they improve recall within a configurable FP budget or strictly reduce FP without harming FN. This division of labor yields compact rule sets that preserve decisive tokens, offer line level justifications, and run at CPU only, low latency inference. Empirically, across real industrial scenarios of Huawei Cloud Core Network, *Smart Eye* attains superior F1 while producing explanations that align with operator mental models and runbook triggers. Qualitatively, the staged evidence chain transforms each alert into a minimal witness optionally contextualized by corroborating fields. *Smart Eye* extends the score latency frontier: it reaches higher accuracy than embedding heavy baselines without their inference costs.

6 ACKNOWLEDGMENTS

This work was supported by the National Key R&D Program of China (Young Scientist Program) (Grant No. 2025YFB3110000) and the National Natural Science Foundation of China-Research Grants Council (RGC) Joint Research Scheme (62321166652).

References

- [1] Alibaba. 2025. Alibaba Cloud Health Dashboard. <https://status.aliyun.com/#/historyEvent>.
- [2] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, et al. 2024. Automatic root cause analysis via large language models for cloud incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems*. 674–688.
- [3] Zeming Chen, Qiyue Gao, and Lawrence S Moss. 2021. NeuralLog: Natural language inference with joint neural and logical reasoning. *arXiv preprint arXiv:2105.14167* (2021).
- [4] Hang Cui, Jingjing Li, Haotian Si, Quan Zhou, Changhua Pei, Gaogang Xie, and Dan Pei. 2025. TShape: Rescuing Machine Learning Models from Complex Shapelet Anomalies. In *2025 IEEE 36th International Symposium on Software Reliability Engineering Workshops (ISSREW)*. 9–14. doi:10.1109/ISSREW67781.2025.00038
- [5] Tianyu Cui, Shiyu Ma, Ziang Chen, Tong Xiao, Chenyu Zhao, Shimin Tao, Yilun Liu, Shenglin Zhang, Duoming Lin, Changchang Liu, et al. 2025. LogEval: A comprehensive benchmark suite for LLMs in log analysis. *Empirical Software Engineering* 30, 6 (2025), 173.
- [6] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.
- [7] Evgeniy Gabrilovich and Shaul Markovitch. 2004. Text categorization with many redundant features: Using aggressive feature selection to make SVMs competitive with C4.5. In *Proceedings of the twenty-first international conference on Machine learning*. 41.
- [8] Google. 2023. Google Cloud Services Hit by Outage in Paris. <https://thenewstack.io/google-cloud-services-hit-by-outage-in-paris/>.
- [9] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tieqiao Zheng, Bo Zhang, Junran Peng, and Qi Tian. 2024. Logformer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 135–143.
- [10] Haixuan Guo, Shuhan Yuan, and Xintao Wu. 2021. Logbert: Log anomaly detection via bert. In *2021 international joint conference on neural networks (IJCNN)*. IEEE, 1–8.
- [11] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2025. Weakly-Supervised Log-Based Anomaly Detection with Inexact Labels via Multi-Instance Learning. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering* (Ottawa, Ontario, Canada) (ICSE '25). IEEE Press, 2918–2930. doi:10.1109/ICSE55347.2025.00189
- [12] Shilin He, Jieming Zhu, Pinjia He, and Michael R Lyu. 2016. Experience report: System log analysis for anomaly detection. In *2016 IEEE 27th international symposium on software reliability engineering (ISSRE)*. IEEE, 207–218.
- [13] Junjie Huang, Zhihan Jiang, Zhuangbin Chen, and Michael Lyu. 2025. No More Labelled Examples? An Unsupervised Log Parser with LLMs. *Proc. ACM Softw. Eng.* 2, FSE, Article FSE107 (June 2025), 24 pages. doi:10.1145/3729377
- [14] Yintong Huo, Yichen Li, Yuxin Su, Pinjia He, Zifan Xie, and Michael R Lyu. 2023. Autolog: A log sequence synthesis framework for anomaly detection. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 497–509.
- [15] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403* (2024).
- [16] Max Landauer, Sebastian Onder, Florian Skopik, and Markus Wurzenberger. 2023. Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications* 12 (2023), 100470.
- [17] Max Landauer, Florian Skopik, and Markus Wurzenberger. 2024. A critical review of common log data sets used for evaluation of sequence-based anomaly detection techniques. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1354–1375.
- [18] Jinyang Liu, Junjie Huang, Yintong Huo, Zhihan Jiang, Jiazhen Gu, Zhuangbin Chen, Cong Feng, Minzhi Yan, and Michael R Lyu. 2023. Log-based Anomaly Detection based on EVT Theory with feedback. *arXiv preprint arXiv:2306.05032* (2023).
- [19] Siyang Lu, Xiang Wei, Yandong Li, and Liqiang Wang. 2018. Detecting anomaly in big data system logs using convolutional neural network. In *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*. IEEE, 151–158.
- [20] Weibin Meng, Ying Liu, Yichen Zhu, Shenglin Zhang, Dan Pei, Yuqing Liu, Yihao Chen, Ruizhi Zhang, Shimin Tao, Pei Sun, et al. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs.. In *IJCAI*, Vol. 19. 4739–4745.
- [21] Microsoft. 2025. Microsoft's Azure networking takes a worldwide tumble. https://www.theregister.com/2024/07/30/microsofts_azure_portal_outage/.
- [22] Xiaohui Nie, Hang Cui, Changhua Pei, Haotian Si, Ke Xiang, Jingjing Li, Yanbiao Li, Gaogang Xie, and Dan Pei. 2025. DeST: An Unsupervised Decoupled Spatio-Temporal Framework for Microservice Incident Management. In *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*. 335–346. doi:10.1109/ISSRE66568.2025.00042
- [23] Changhua Pei, Zihan Liu, Jianhui Li, Erhan Zhang, Le Zhang, Haiming Zhang, Wei Chen, Dan Pei, and Gaogang Xie. 2024. Self-evolutionary group-wise log parsing based on large language model. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 49–60.
- [24] Jiaxing Qi, Shaohan Huang, Zhongzhi Luan, Shu Yang, Carol Fung, Hailong Yang, Depei Qian, Jing Shang, Zhiwen Xiao, and Zhihui Wu. 2023. Loggpt: Exploring chatgpt for log-based anomaly detection. In *2023 IEEE International Conference on High Performance Computing & Communications, Data Science & Systems, Smart City & Dependability in Sensor, Cloud & Big Data Systems & Application (HPCC/DSS/SmartCity/DependSys)*. IEEE, 273–280.
- [25] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789* (2023).
- [26] Komal Sarda, Zakeya Namrud, Marin Litoiu, Larisa Shwartz, and Ian Watts. 2024. Leveraging large language models for the auto-remediation of microservice applications: An experimental study. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 358–369.
- [27] Yicheng Sui, Yuzhe Zhang, Jianjun Sun, Ting Xu, Shenglin Zhang, Zhengdan Li, Yongqian Sun, Fangrui Guo, Junyu Shen, Yuzhi Zhang, et al. 2023. Logkg: Log failure diagnosis through knowledge graph. *IEEE Transactions on Services Computing* 16, 5 (2023), 3493–3507.
- [28] Yongqian Sun, Shiyu Ma, Tong Xiao, Yongxin Zhao, Xuhui Cai, Wei Dong, Yue Shen, Yao Zhao, Shenglin Zhang, Jing Han, et al. 2025. Accurate and Interpretable Log-Based Fault Diagnosis using Large Language Models. *IEEE Transactions on Services Computing* (2025).
- [29] Yongqian Sun, Tinghua Zheng, Xidao Wen, Weihua Kuang, Heng Liu, Shenglin Zhang, Chao Shen, Bo Wu, and Dan Pei. 2025. A Multimodal Intelligent Change Assessment Framework for Microservice Systems Based on Large Language Models. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. 378–388.
- [30] Robert West, Evgeniy Gabrilovich, Kevin Murphy, Shaohua Sun, Rahul Gupta, and Dekang Lin. 2014. Knowledge base completion via search-based question answering. In *Proceedings of the 23rd international conference on World wide web*. 515–526.
- [31] Pei Xiao, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. 2024. LogCAE: An Approach for Log-based Anomaly Detection with Active Learning and Contrastive Learning. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. 144–155. doi:10.1109/ISSRE62328.2024.00024
- [32] Lin Yang, Junjie Chen, Zan Wang, Weijing Wang, Jiajun Jiang, Xuyuan Dong, and Wenbin Zhang. 2021. Plog: Semi-supervised log-based anomaly detection via probabilistic label estimation. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 230–231.
- [33] Zhenhe Yao, Haowei Ye, Changhua Pei, Guang Cheng, Guangpei Wang, Zhiwei Liu, Hongwei Chen, Hang Cui, Zeyan Li, Jianhui Li, et al. 2024. SparseRCA: Unsupervised Root Cause Analysis in Sparse Microservice Testing Traces. In *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 391–402.
- [34] Boxi Yu, Jiayi Yao, Qiulai Fu, Zhiqing Zhong, Haotian Xie, Yaoliang Wu, Yuchi Ma, and Pinjia He. 2024. Deep learning or classical machine learning? an empirical study on log-based anomaly detection. In *Proceedings of the 46th IEEE/ACM international conference on software engineering*. 1–13.
- [35] Delvin Ce Zhang and Hady Lauw. 2022. Dynamic topic models for temporal document networks. In *International Conference on Machine Learning*. PMLR, 26281–26292.
- [36] Delvin Ce Zhang and Hady W Lauw. 2021. Semi-supervised semantic visualization for networked documents. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 762–778.
- [37] Shenglin Zhang, Yuhe Ji, Jiaqi Luan, Xiaohui Nie, Ziang Chen, Minghua Ma, Yongqian Sun, and Dan Pei. 2024. End-to-end autolm for unsupervised log anomaly detection. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1680–1692.
- [38] Shenglin Zhang, Dongwen Li, Zhenyu Zhong, Jun Zhu, Minghan Liang, Jiexi Luo, Yongqian Sun, Ya Su, Sibao Xia, Zhongyou Hu, et al. 2022. Robust system instance clustering for large-scale web services. In *Proceedings of the ACM Web Conference 2022*. 1785–1796.
- [39] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 807–817.

A Related work

Research on log anomaly detection has progressed along three intertwined lines: (i) sequence models over parsed log keys, (ii) neural representation learning from raw or parsed logs, and (iii) pretrained and adapt transformers pipelines. We briefly review representative systems in each family and clarify how our approach differs.

Sequence models with template parsing. DeepLog[6] established the modern parse, sequence model and next event prediction recipe. Logs are first parsed into templates and parameters; an LSTM then learns normal transition patterns and flags unlikely events. The strength of this paradigm is its ability to model temporal dependencies in service workflows. Subsequent variants retain this overall structure while replacing the backbone or the scoring rule. Related methods such as LogAnomaly[20] extend the sequential view with quantitative deviations in parameters, improving sensitivity to value shifts but still inheriting the parser’s failure modes. Their weakness is brittle dependence on parsing quality small lexical cues that operators regard as decisive can be normalized away by parsers, leading to missed alerts.

Convolutional feature learners. LogCNN[19] applies convolutional encoders to capture local n gram style regularities in tokenized logs, a choice that improves inference throughput over recurrent models and can be robust to moderate template drift. LightAD[34] targets deployment efficiency via compact architectures and streamlined features, prioritizing low latency while trading some headroom in accuracy on heterogeneous, real-world corpora. Both methods exemplify the speed and accuracy space we analyze empirically.

Robustness oriented pipelines. RobustLog[39] emphasizes resilience to noise and template variability in industrial logging, typically via stronger denoising, feature selection, or rule stabilization before the learner. Such systems aim to reduce false positives under drift but continue to funnel evidence through embeddings that are hard to audit line by line. Our results include RobustLog as a robustness-focused baseline on the same splits and settings.

Pretrained transformers for logs. LogBERT[10] imported masked token pretraining to the log domain, yielding strong generalization across services and vendors; LogFormer[9] extended this line with a pretrain and tune pipeline tailored to log semantics. These transformer baselines typically deliver higher F1 than shallow encoders but at the cost of heavier inference and reduced transparency, since decisions emerge from dense embeddings rather than explicit textual evidence. Both are among the strongest neural baselines in our study.

Neural symbolic and prompt driven detectors. NeuralLog[3] represents a hybrid line of work that blends neural encoders with logical operators; when adapted to logs, the goal is to capture rule like structure while retaining statistical flexibility. PLELog[32] reflects a complementary trend that leverages prompt or pattern learning to better align detectors with human readable cues. However, in practice their alarms still pass through distributed representations, limiting per line auditability. Each appears in our baseline panel with paper-recommended settings.

Unlike the above families which either depend on fragile parsing or hide decisions behind embeddings, we treat LLMs strictly as

proposal engines and elevate a deterministic verifier to arbiter. Concretely, we retrieve compact key textual witnesses under an explicit false positive and false negative budget, then monotonically refine them via specialization and exclusions; the resulting detector quotes the exact tokens that triggered the alert and runs at CPU-only, low latency. This division of labor preserves decisive lexical cues and yields human auditable evidence chains while matching or exceeding the accuracy of embedding heavy baselines on industrial corpora.

B Hyperparameter sensitivity

We make the three acceptance factors explicit so their roles and interactions are transparent.

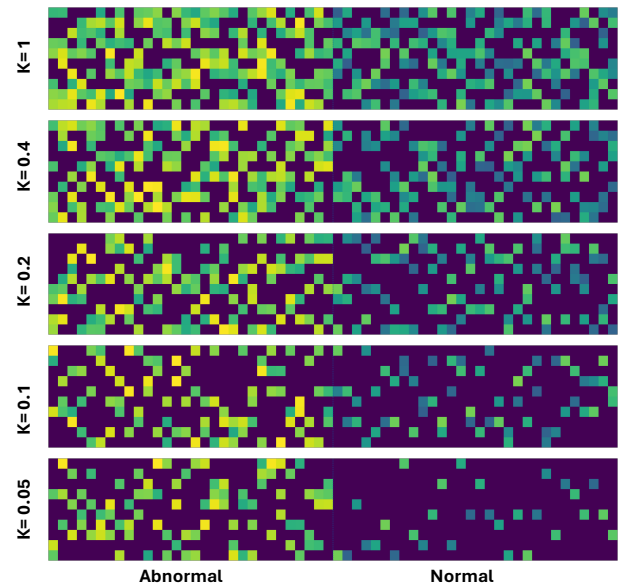


Figure 8: Attention heatmaps for κ_1 robustness testing

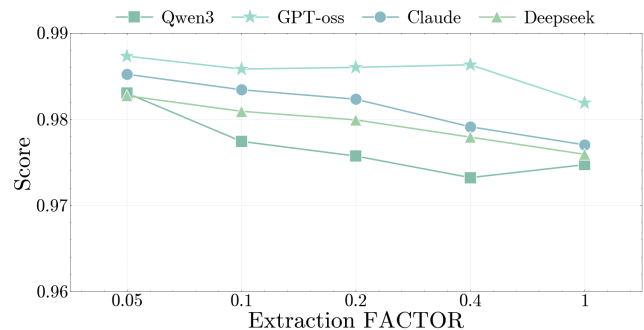


Figure 9: Robustness testing for κ_1 extraction

Stage I Pattern Extraction (κ_1). A candidate extraction pattern p is accepted when it improves recall without disproportionate false

positives (FP). Operationally, we require $\Delta FN \leq 0$ and bound the tolerated FP increase by the dropped FN:

$$\Delta FN \leq \kappa_1 \Delta FP$$

Smaller κ_1 is more conservative; larger κ_1 accelerates recall growth but can inflate intermediate FP counts before aggregation and refinement.

Sensitivity to κ_1 (Pattern Extraction). Figure 9 summarizes the curves (values as provided in the figure data). We sweep $\kappa_1 \in \{0.05, 0.1, 0.2, 0.4, 1.0\}$. Across model families (Qwen3, GPT-oss, Claude, DeepSeek), macro-F1 exhibits a broad plateau for $\kappa_1 \in [0.1, 0.4]$, with mild degradation at the extremes. Consistently, the attention maps in Figure 8 show sharper focus on decisive fields in this mid range, while overly large κ_1 diffuses attention to non discriminative context, explaining the observed precision drift.

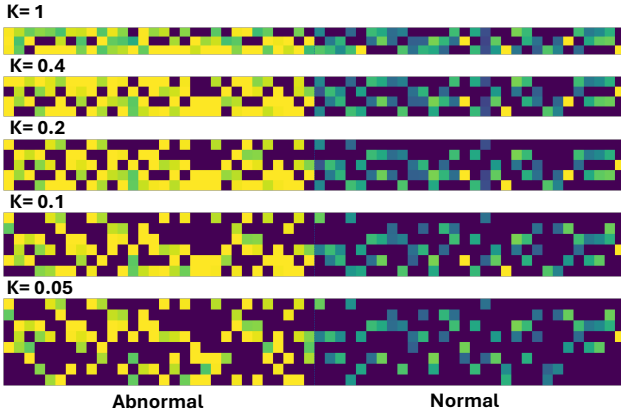


Figure 10: Attention heatmaps for κ_2 robustness testing

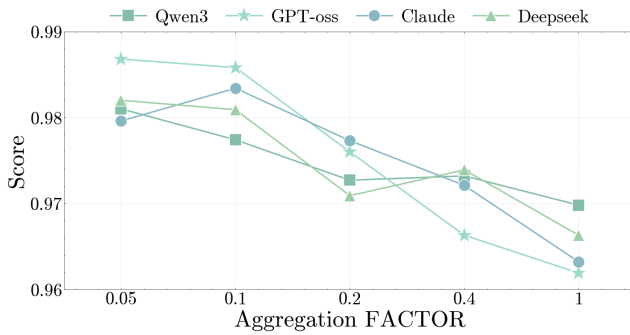


Figure 11: Robustness testing for κ_2 aggregation

Stage I Pattern Aggregation (budget κ_2). Aggregation considers broader parent patterns that unify several children under a budget. We apply the same guarded trade-off:

$$\Delta FN \leq \kappa_2 \Delta FP,$$

and also enforce a per volume cap so that cumulative FP growth per 100k normal lines never exceed ϵ violently in a round. This prevents recall gains from being realized via a single overly permissive parent.

Sensitivity to κ_2 (Pattern Aggregation). We sweep $\kappa_2 \in \{0.05, 0.1, 0.2, 0.4, 1.0\}$. Aggregation is slightly more brittle than extraction (Figure 11): permissive settings compress many children into a parent at the cost of measurable precision loss on noisier services; conservative settings maintain precision but leave some recall unrealized until Stage II. The heatmaps in Figure 10 corroborate this trend: tighter budgets concentrate attention on parent defining tokens, whereas permissive κ_2 spreads attention onto irrelevant fields, aligning with the precision drop.

C Robustness to In-Context Example Count

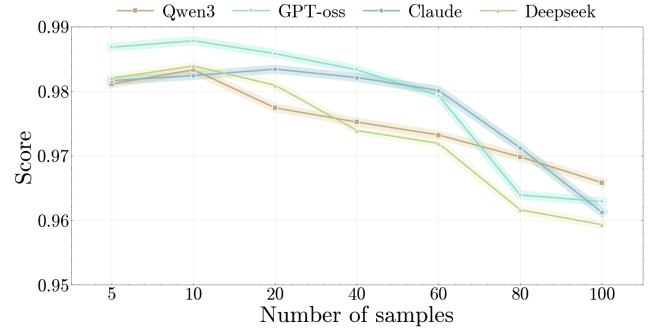


Figure 12: Effect of in context example count m on F1 score across different models.

We study how the number of in context abnormal examples per proposal round (m) affects *Smart Eye*. Unless noted, we reuse the chronological splits, metrics, and implementation in 4 and fix hyperparameters at the recommended defaults from §B. For each $m \in \{5, 10, 20, 40, 60, 80, 100\}$, we prompt four LLMs (Qwen3, GPT-oss, Claude, Deepseek) with m label balanced exemplars drawn from the same round's positives and hard negatives, keeping proposal batch size and temperature unchanged.

Increasing m beyond 40 reduces F1 for every model (e.g., GPT-oss drops from 0.9878 at $m = 10$ to 0.9629 at $m = 100$), consistent with prompt bloat, redundant or conflicting exemplars, and weaker inductive bias once salient contrasts are diluted. The inter-model spread near optimum is small, but variance widens at large m , indicating sensitivity to example quality when the prompt is crowded.

These studies indicate that our method is robust and deployment ready in real industrial settings. The κ_1 and κ_2 sweeps reveal a wide operating plateau. Meanwhile moderate extraction and aggregation strictness maintains macro-F1 within a narrow band across model families. Varying the number of in context examples, our method has always remained within a very high, industrially acceptable range of F1 scores, which demonstrates favorable accuracy and interpretability, supporting reliable operation in industrial environments.